Comparison: teammates' architecture and opinions vs this implementation

This document compares the two teammate notes in `teammate/` with what `riskdep` implements today, as described in `ARCHITECTURE.md`, `ANALYSIS_MONITORING.md`, `NEXT_STEP.md` and the code under `riskdep/`.

- **Team 1** (`teammate/team1.md`, in French): two AI conversations.
  - A ChatGPT thread on the product vision, real-time alerting, a self-hosted stack, a typology of dependencies and parallels with other fields.
  - A Claude thread proposing a TypeScript stack and a Vault V2 spec v0.1.
- **Team 2** (`teammate/team2.md`): a Claude conversation that reviews the original plan (graph, impact, monitoring, execution) against the Belem track's scoring grid. It adds a decision layer, generic discovery, and an exit / run / fixed-oracle model.

Contents

- 1. Summary
- 2. Side-by-side
- 3. Where the teammates and the implementation agree
- 4. Where the implementation chose differently
- 5. Teammate ideas not implemented yet
- 6. What the implementation has that neither note covers
- 7. Corrections to the teammate notes
- 8. Questions to settle as a team

1. Summary

Theme	Team 1	Team 2	Implementation
Product focus	ChatGPT: simulator first, then "monitoring must be P0". Claude: limits and exit first, scenario library before Monte Carlo	Score-driven: discovery (30 %), impact (30 %), **decision** (25 %), demo (15 %)	Three things: a STAY / REDUCE / EXIT verdict (`analyze`), block-by-block alerts (`watch`), a dependency graph (`graph`)
Language	ChatGPT: Python end to end. Claude: TypeScript end to end	Not discussed	**Python** CLI (`requests`, `numpy`, `networkx`, `eth-abi`)
Morpho version	Claude: **Vault V2** (V1 `maxWithdraw` returns 0 on V2)	MetaMorpho v1.1 (`withdrawQueue`, `lostAssets`)	**Vault V2** and Blue; V1 is an opaque layer

Theme	Team 1	Team 2	Implementation
Graph	Dependency typology; look-through, blast radius, concentration	Typed nodes; **edges carry a transmission function**; DebtRank	Typed nodes (Actor, Pool, Asset, Oracle, Admin, External) with amounts on edges. Drawn, **not used for propagation** (gap R1)
Impact	Gap shock → bad debt; liquidity at 24 h / 7 d	LaaR with a liquidation cascade and DEX depth; oracle type decides whether liquidations fire	Monte Carlo over layer incidents for one PT position; per-market liquidation curve and unrealised bad debt; no cascade
Decision	Not central	Breakeven probability p*, partial exit, run / queue model, fixed-oracle drain d*	Verdict from carry vs expected loss vs exit cost, CVaR99 tail budget, de-lever sizing, "risk multiple to exit"
Monitoring	ChatGPT: WebSocket, event-driven, fast / deep engines	Two speeds: topology on events, state in batches	Polling every 12 s, events plus one batched state read per batch, refresh every 300 blocks
Alerts delivery	Telegram / ntfy	Not discussed	Terminal and JSONL only (gap D5)
Execution	Not discussed	**Cut it**, use Multicall3 only for reads	Not implemented, documented as X1 with a design
Data sources	ChatGPT: no SaaS, own node eventually. Claude: chain only, GraphQL only as a dev shortcut	Morpho GraphQL for speed, RPC to verify; aggregator quotes	Keyless public sources. **"The APIs find, the chain decides"**: Morpho, Pendle and DefiLlama APIs are used for discovery, and amounts are read on-chain
UI	ChatGPT: Streamlit + Cytoscape. Claude: React Flow, static site, simulator in the browser	Clickable node, failure-mode picker	Static HTML reports; graph in Cytoscape.js; no interactive shock sliders

Short reading. The implementation is closest to **Team 2's graph model** (same node types) and to **Team 1 / ChatGPT's conclusion that event monitoring comes first** (two of the three modes are watchers). Its main gaps compared with the notes are:

- the **generic graph-based propagation** (Team 2's transmission functions, DebtRank; Team 1's look-through and blast radius);
- the **run / queue model** of the exit (Team 2);
- **push alerts** (both teams).

2. Side-by-side

2.1 Scope and target

	Team 1	Team 2	Implementation
Who uses it	A fund such as Belem: risk officer and allocator	A fund manager deciding to stay or exit	A risk or portfolio manager (`analyze`), a depositor (`watch --vault`), an address with positions (`watch --user`)
Root input	Fund addresses or a synthetic portfolio of public V2 vaults	The position (vault shares), separate from the vault	One PT position (`analyze`), one vault plus a depositor, or one address
Protocols	Morpho only, with a warning about giving a false impression of global coverage	Morpho + Pendle + Ethena chain; Uniswap called a weak liquidity assumption	Morpho Blue + Vault V2, Pendle for PT collateral; Ethereum mainnet only
Portfolio view	Central: "NAV of the fund", exposure % per root asset	Implicit	**Missing**: one position at a time (gap M6)

2.2 Stack

Decision	Team 1 / ChatGPT	Team 1 / Claude	Implementation	Co
Language	Python	TypeScript (viem, `@morpho-org/blue-sdk-viem`)	Python	Pyt kep Mo ma hea sha acc is r imp fro pin sou `li` tha fro SDI

Decision	Team 1 / ChatGPT	Team 1 / Claude	Implementation	Co
Graph library	NetworkX	React Flow + elk / dagre	NetworkX (in `discover.py`) + Cytoscape.js for rendering	—
Storage	DuckDB / Parquet + PostgreSQL	Snapshot JSON versioned in git	JSON request cache (`data/snapshot/`), recorded replays (`data/replay/`), JSONL alerts, cursor files	Clo Tea Cla "sn a c and tha
Simulation	NumPy / SciPy, deterministic first	Pure function of the price vector; Monte Carlo as a thin loop on top	NumPy Monte Carlo, 20,000 months, seeded	—
UI	Streamlit	Vite + React + Tailwind	Static HTML	No inte
Deployment	Docker Compose	Static site + GitHub Actions cron	Local CLI	No (ga

2.3 Dependency graph

	Team 1	Team 2	Implementation
Node types	Structural / economic / behavioural dependencies; issuers, custodians, bridges, multisigs	Position, Vault, Market, Asset (with `kind`), Oracle, Admin, Chain / Bridge, Off-chain entity	Actor (address, Vault V2, V1, main lenders), Pool (Blue market, Pendle pool), Asset, Oracle (decomposed into feeds), Admin, External. `analyze` has its own layers: chain, issuer, AMM, oracle, lending, stable, off-chain
Holders and borrowers	Common holders across protocols; behavioural dependencies	Kept as **distributions** on markets, not as nodes; only whales shown	Borrowers kept as data (`market_risk.py`), not as nodes; main lenders (≥ 5 % of supply) **are** nodes

	Team 1	Team 2	Implementation
Edges	Each relation needs its own propagation semantics	Each edge type has a **transmission function** (`allocates_to`, `collateral`, `loan_asset`, `wraps`, `priced_by`, `controlled_by`, `issued_by`…)	Edges carry amounts and shares; propagation semantics live in `propagate.py`, per **layer**, not per edge
Shared nodes	Single points of failure, accumulation risk (insurance analogy)	—	Visible in the graph (for example one `RLUSD / USD` feed under all 10 Sentora markets), **not measured or alerted** (gap R3)
Propagation limits	—	DebtRank, an ε floor, terminal nodes	A fixed layer list in `analyze` acts as a depth limit; no general rule (gap R2)

2.4 Impact model

	Team 1	Team 2	Implementation
Shock type	Named scenarios (ETH −40 %, USDC depeg, combined). Claude: jumps, not diffusion	A small set of failure modes: `price_drop`, `depeg`, `freeze`, `total_loss`, `oracle_misprice(±x)`, `oracle_stale`	Layer events: `chain`, `issuer_hack`, `issuer_depeg` (with recovery), `amm`, `oracle` (downward misreport), `lending`
Probabilities	Claude: scenarios first, a jump mixture with entered probabilities later	Avoid pretending to know them: use a **breakeven probability**	**Base rates** from DefiLlama's hack record by size, category and age; depegs from `registry.json`, each with a written basis
Liquidation	Gap shock gives bad debt directly; slippage later	LIF formula, liquidation loop against DEX depth (LaaR)	LIF formula; per-market curve at −2 / −5 / −10 / −20 % over **every borrower**; no price-impact loop

	Team 1	Team 2	Implementation
Oracle behaviour	Oracle vs independent price is "critical"	Classify every oracle as market-tracking or not; the gap to the market price is the main signal	Inferred by comparing prices (market-tracking vs fundamental, "ambiguous" near par, with a sensitivity run); oracle vs DefiLlama, warn from 3 %
Slippage	—	Aggregator quotes at several sizes, square-root fit	Real **Pendle** quotes for PTs, including a partial re-quote; other collateral is not quoted (gap M3)

2.5 Decision and exit

	Team 1	Team 2	Implementation
Output	Withdrawable now, after 24 h, after 7 d	p* = exit cost / severe loss; partial exit at marginal slippage = marginal loss; top paths; the signal that would change the decision	STAY / REDUCE / EXIT; reasons with the top drivers; "exit only pays if probabilities were k× the estimates" (`risk_multiple_to_exit`, the multiplicative analogue of p*); de-lever sized to survive a stress drop
Partial exit	—	Sized at the optimum point	Sized to reach a **stress target** (`sale_to_reach`), not at the marginal optimum
Tail	—	CVaR	CVaR99 against a tail budget in `registry.json`
Vault exit capacity	`maxWithdraw` (V1 thinking), corrected to "use the SDK" for V2	W_0 = Σ min(supply, idle) over the withdraw queue	V2-correct: plain withdrawal = idle + the liquidity adapter's market; `forceDeallocate` reach is reported separately, at its penalty
Run / queue	Liquidity at 24 h / 7 d under scenarios	φ, loss as a function of queue position, depositor concentration	**Missing**. "Large holders of a vault" is listed as missing in `ANALYSIS_MONITORING.md` §3
Fixed-oracle drain	"Resolv-type" scenario named as key	d* ≈ 1 − LLTV + buy cost; Public Allocator `maxIn` and cap headroom increase what can be drained	Oracle mode is inferred, and a non-tracking oracle means no liquidation in the model. **No d* threshold and no Public Allocator**

	Team 1	Team 2	Implementation
Stuck time	—	AdaptiveCurveIRM path at 100 % utilisation	Utilisation ≥ 99 % warn; borrow APY +50 % warn; no stuck-time estimate

2.6 Monitoring and alerts

	Team 1 / ChatGPT	Team 2	Implementation
Transport	WebSocket + reconciliation by polling	—	Polling (`docs/websocket-vs-polling.md` documents the choice); the WS hybrid is planned (next step 6)
Cadence	Event processor + incremental risk engine	Topology on events, state every block or minute through Multicall3	Events + **one batched state read per batch at one block**; topology and lenders refreshed every 300 blocks
Fast vs deep engine	Separate them	—	In practice yes: `watch` is the fast path, `analyze` the deep one. They do not share a graph (gap R1)
Signals	Depeg (oracle vs market), LLTV / oracle change, liquidatable spike, `maxWithdraw` drop, allocation concentration, admin / timelock changes, utilisation	Oracle gap, pending timelocked changes, whale HF, withdrawable liquidity	**All covered** except whale behaviour on the vault side: timelocked `Submit` decoded with its date; zero-timelock risk functions; upgrades (EIP-1967), owners, aggregators, pauses, outsized mints; unrealised bad debt; liquidation curve; exit capacity; lender concentration
Delivery	Telegram / ntfy, severity levels, deduplication	—	Severity levels yes; **no push and no deduplication across restarts** (D4, D5)
Reliability	Node redundancy, stale-data detection, catch-up	—	Block-hash cross-check against a second RPC; API vs chain cross-check; cursor saved last; gaps reported, not backfilled

2.7 Discovery

Layer	Team 2 proposal	Implementation

Layer	Team 2 proposal	Implementation
Generic probes	ERC-4626 `asset()`, Chainlink interface, EIP-1967 slots, `owner()`, Safe owners and threshold, Timelock `getMinDelay`, ABI-driven crawl of zero-argument address getters	EIP-1967 slots, `owner()`, Chainlink aggregators, Morpho `ChainlinkOracleV2` getters (`BASE_FEED_1`, `BASE_VAULT`…). **No** ABI-driven crawl, no Safe threshold, no recursive ERC-4626 unwrapping
Protocol adapters	Morpho Blue, MetaMorpho, Morpho oracle, Pendle	Morpho Blue, Vault V2 + `MorphoMarketV1AdapterV2`, Morpho oracle, Pendle
Curated registry	YAML of issuers, custodians, bridges	`registry.json`: issuers with depeg priors, thresholds, each with its basis (4 yield issuers by hand, gap M7)
LLM fallback	Stretch	—

3. Where the teammates and the implementation agree

1. **Morpho as the POC.** Both notes and the implementation chose it.
2. **Vault V2.** Team 1 / Claude and the implementation agree, and both make the point that V1 accounting (`maxWithdraw`, `lostAssets`) does not apply.
3. **Node types.** Team 2's types (Vault/users, Pool, Asset, Oracle, Admin) are those of the `graph` command. Vaults are modelled as Actors, a choice argued in `ANALYSIS_MONITORING.md` §1.
4. **Borrowers as data, not nodes.** Team 2 recommends it; `market_risk.py` does it.
5. **Oracle behaviour is central.** All three name the oracle that does not follow the market as the key failure. The implementation infers the oracle mode, runs a sensitivity case and watches the gap to DefiLlama.
6. **Two-speed monitoring.** Events tell you *that* something changed; a batched state read tells you *what* changed. Proposed by Team 2 and ChatGPT, done in `poller.py` and the readers.
7. **Admin and timelock watching.** Matches Team 2's "if the timelock is longer than your reaction time, a malicious change costs you an exit, not a total loss": zero-timelock risk functions are flagged, and a `Submit` for one is critical.
8. **No paid data vendor.** ChatGPT and Team 1 / Claude both want it. The implementation uses keyless public sources and treats the RPC as an interchangeable provider.
9. **Explicit metrics over a single score.** ChatGPT: "managers must understand why an alert exists". Every alert carries its reason, and every threshold its basis in `registry.json`.
10. **Execution deferred.** Team 2 says cut it; the implementation documents it as X1 with a security design, not code.

4. Where the implementation chose differently

Topic	Teammate position	Implementation	Assessment

Topic	Teammate position	Implementation	Assessment
Language	Team 1 / Claude: TypeScript to reuse the Morpho SDK	Python	Reasonable. The cost is re-implementing SDK maths; tests mitigate it. Revisit only for a React UI.
Probabilities	Team 2: avoid them, report a breakeven p*. Team 1 / Claude: scenarios first, Monte Carlo later	Base-rate probabilities and Monte Carlo are central to `analyze`	The verdict relies on base rates (gap M1). The `risk_multiple_to_exit` line partly answers Team 2 by saying how wrong the probabilities would have to be. Showing a scenario table alongside would satisfy both notes.
Scope of `analyze`	Team 2: any vault, many markets, aggregated	One PT borrower position	The Belem use case (a fund *depositing* in vaults) is served by `watch --vault`, not by `analyze`. There is no loss number for a vault depositor under a scenario. **This is the largest gap against the track's "impact" criterion.**
Graph as engine	Team 2: one graph with generic propagation	The graph is a rendering; rules use their own structures	Documented as gap R1 (L). It limits the "adding a protocol = one adapter" claim Team 2 wanted to demo.
Liquidity source	Team 2: an aggregator, generic for any token	Pendle API only	Next step 2 needs it; NEXT_STEP notes that no keyless aggregator has been verified.
Transport	ChatGPT: WebSocket	Polling	Argued in `docs/websocket-vs-polling.md`; the hybrid is planned. Latency is 25–40 s.
UI	Both: interactive (sliders, click a node, pick a shock)	Static reports	Weak for the demo (15 % of the score).

4.1 Language: Python instead of TypeScript

What was proposed. Team 1's Claude thread argues for TypeScript end to end: viem for reads and multicall, `@morpho-org/blue-sdk-viem` for live market and position data, and `@morpho-org/morpho-sdk` for Vault V2, so that accrual, LTV and health maths come from Morpho's own code; one language from reading to the UI. Team 1's ChatGPT thread argued the opposite: Python

for simulation, graphs and backtesting, TypeScript only if an indexer such as Ponder brings a measurable gain.

What the implementation does. Python, with the maths re-implemented where needed:

- Blue's shares-to-assets conversion (virtual shares and assets);
- the liquidation incentive factor, the health factor, the bad debt of a position;
- the Vault V2 cap identifiers and the withdrawal limits.

Two pieces are deliberately *not* re-implemented: the vault's accrual (it reads `totalAssets()`, which the contract computes) and the interest rate (it calls the market's IRM `borrowRateView` on-chain).

What it costs. Every re-implemented formula can drift from Morpho's. Today the safeguards are the cross-checks against the Morpho API (health factor within 0.02 % on the demo, vault total assets) and the tests.

What could be implemented.

- **SDK parity check** (S): a small Node script using `@morpho-org/blue-sdk-viem` that computes health factor, debt and withdrawal limits for the recorded positions, compared with `riskdep`'s numbers in a test. It keeps Python while borrowing the SDK as a reference.
- **TypeScript only at the edge** (M, if a React UI is built): the UI consumes `report.json` and `graph.json`, which are already a stable data contract, so no rewrite of the engine is needed.

4.2 Probabilities: base rates and Monte Carlo instead of a breakeven probability

What was proposed. Team 2 warns that probabilities are "the weakest part of any such model" and proposes the **breakeven probability** p* = exit cost ÷ L(severe case): exit if you believe the severe case is more likely than p*. Team 1's Claude thread asks for named scenarios first and Monte Carlo later; its ChatGPT thread lists the trade-off: deterministic is explainable, Monte Carlo gives distributions but rests on hard-to-calibrate assumptions.

What the implementation does. `analyze` estimates a yearly probability per layer from DefiLlama's hack record (by protocol size and category) and from registry entries for depegs, runs 20,000 simulated months, and decides from the expected loss, the CVaR 99 % and the exit cost. Its line "exit would only pay if incident probabilities were N× the estimates" (`risk_multiple_to_exit`) is a global version of Team 2's idea: how wrong the probabilities must be to change the decision.

What it costs. The verdict inherits the weakness of base rates (gap M1), and a reader cannot see the loss of one specific scenario.

What could be implemented.

- **A scenario table in the `analyze` report** (S): for named shocks (yield-token depeg of 5, 10 and 20 %, oracle misprice of −10 %, Pendle exploit of 30 %, …), the deterministic loss L(s) is one call to the existing pure function `propagate.loss()`. Next to each: whether it liquidates, and p*(s) = exit cost ÷ L(s). The Monte Carlo stays, and the two views answer both notes.
- **Scenarios as the default verdict basis** (S, a decision to take as a team): decide from "the worst named scenario you believe above p*" and keep the Monte Carlo as context.

4.3 Scope of `analyze`: one borrower, not a vault depositor

What was proposed. Team 2 frames the use case as the track's: a fund *depositing* in vaults, with the loss aggregated over every market the vault allocates to (Σ over markets of the vault's share of supply × bad debt), and an exit cost that comes from withdrawal liquidity, not from a swap.

What the implementation does. `analyze` evaluates one Pendle PT position used as Morpho Blue collateral, from the borrower's side. The depositor's side is covered by `watch`: exposure, exit capacity, unrealised bad debt now, and each market's liquidation curve. No mode answers "how much do I lose if X happens?" for a depositor.

What it costs. It is the largest gap against the track's "impact" criterion (30 % of the score).

What could be implemented. Most of the pieces exist:

- **`analyze --vault <address> [--user <address>]`** (M):
  - for a scenario (a price drop on one collateral asset, on every market using it, or on one oracle feed), apply the drop in `market_risk.value()`, which already returns each market's bad debt at a given fall;
  - multiply by the vault's share of each market's supply (`market_risk.vault_supply_shares`), sum over markets, and divide by the depositor's share of the vault;
  - report the loss next to the exit capacity now (`VaultState.withdrawable` and `force_withdrawable`), and p* = exit cost ÷ loss.
- **Limits to state in its output:** liquidations are assumed to clear at the oracle price with the liquidation bonus, so there is no slippage and no cascade (see 4.5 and the cascade loop in §5); the vault's version decides who bears the loss (a Vault V2 realises it for every holder at the next accrual).

4.4 The graph: a rendering, not the engine

What was proposed. Team 2 makes the graph the engine. Each edge type carries a **transmission function** (how a shock on the child becomes a loss on the parent): `allocates_to` passes the vault's share of a market's bad debt; `collateral` is zero until the LLTV buffer is used up, then non-linear; `priced_by` depends on the failure mode (overpricing, underpricing, freeze). Shocks spread with **DebtRank** (Battiston et al., 2012): each node passes a shock once (which handles cycles), paths below ε of the position are pruned, and terminal nodes stop the spread. Adding a protocol then means producing edges of existing types. Team 1 asks for the same in other words: look-through exposure and blast radius.

What the implementation does. `graph.py` builds typed nodes and edges carrying amounts, from live data, and draws them. The alerts and the `analyze` model work on their own structures; `propagate.py` is a hand-written layer model for one PT position (gap R1).

What it costs. The "adding a protocol is one adapter" claim cannot be demonstrated, and shared dependencies (one `RLUSD / USD` feed under all 10 Sentora markets) are drawn but not measured (gap R3).

What could be implemented.

- **Shared-node exposure from `graph.json`** (S): for each node, the sum of the amounts on the paths from each actor to it, with an alert when one node carries more than a set share of a position. It works on the existing JSON, without a propagation engine.

- **A propagation engine over** `graph.json` (L, built in steps):
  - transmission functions for the three edge types that carry most of the risk: `supplies` (share of the market's bad debt, from `market_risk.value()`), `takes as collateral` (the shock as a price fall), `reads` and `converts through` (a feed or vault-share shock becomes an oracle price shock);
  - DebtRank-style spreading with ε pruning;
  - a function "shock this node with this failure mode, return the loss per actor". It would then serve 4.3 and 4.7.

4.5 Liquidity: Pendle quotes instead of a generic aggregator

What was proposed. Team 2 asks a DEX aggregator (KyberSwap, Odos, 1inch are named) for quotes at several sizes and fits a slippage curve, with square-root price impact as a defensible shape. The same curve serves twice: for how well liquidations clear, and for one's own exit cost. It works for any asset without code per DEX.

What the implementation does. Real quotes from Pendle's API, for the full position and for a partial de-lever, used in the exit cost and the de-lever cost. Other collateral is "not quoted"; liquidators' selling is not modelled.

What it costs. `analyze` cannot price an exit for non-PT collateral, and the bad-debt numbers of `market_risk` assume liquidations clear without price impact.

What could be implemented.

- **A quote source module** (M): probe which aggregators answer without a key (to be verified: none has been tested yet), query 3 or 4 sizes per token, fit impact(q) = k·√q, and cache the curve per token for a few minutes. Fallback to "not quoted" when no source answers.
- **Use it twice** (S once the module exists): in `decide.exit_cost_usd` for any collateral (next step 2), and in `market_risk.value()` to discount the collateral liquidators sell, which makes the bad debt at each fall more realistic and opens the cascade loop listed in §5.

4.6 Transport: polling instead of a WebSocket

What was proposed. Team 1's ChatGPT thread: a WebSocket RPC subscribed to new blocks and logs, event-driven engines, eventually one's own node for detection in seconds.

What the implementation does. Polling every 12 seconds, 2 blocks behind the head, with a saved cursor. The reasons are in `docs/websocket-vs-polling.md`: a WebSocket loses whatever is pushed while it is disconnected and needs the same `eth_getLogs` to fill the gap, while polling never misses a block. The latency is 25 to 40 seconds.

What it costs. For a borrower close to liquidation, 25 to 40 seconds can be too late; for the depositor signals (timelocked changes, unrealised bad debt) it is not.

What could be implemented.

- **Hybrid trigger** (S): subscribe to `newHeads` over the free WebSocket endpoint (verified working) and call the existing `next_batch()` on each head, with the 12-second poll as fallback. The cursor logic stays the source of truth, so nothing can be missed. It removes the wait for the next poll (up to 12 seconds), so the latency becomes about 26 seconds, set by the 2 confirmations, instead of 25 to 40.
- **Zero confirmations** (M): process the head block immediately and roll back alerts on logs flagged `removed` by a reorg. The latency drops to a few seconds, at the cost of alerts that a

reorg can retract.
- **Pending transactions** (M): `newPendingTransactions`, filtered on the watched contracts and simulated with `eth_simulateV1` (both verified on the free node), labelled "pending, not final". It does not see transactions sent through private relays.

4.7 Interface: static reports instead of an interactive tool

What was proposed. Team 1's Claude thread: React Flow with a layered layout, shock sliders computed in the browser, a static site deployable on GitHub Pages. Team 2's demo screen: click a node, pick a failure mode and a severity, and see the loss, the exit cost, p*, the recommendation and the contributing paths. Team 1's ChatGPT thread: Streamlit with Cytoscape.

What the implementation does. A static HTML report for `analyze` and an interactive dependency graph (Cytoscape.js: click a node, filter by type, search). Nothing recomputes a loss in the page.

What it costs. The demo counts for 15 % of the score, and a what-if screen is the most direct way to show discovery, impact and decision together.

What could be implemented.

- **A what-if page on top of `graph.html`** (M), static and without a backend, as Team 1 proposed:
  - `graph.json` gains, per market, the borrower distribution in compact form: collateral value and debt per position, or bucketed by health factor;
  - in the page, picking a node (a collateral asset, an oracle feed) and a failure mode with a severity slider recomputes each market's bad debt with the same formula as `market_risk.value()`, then the loss per actor through the supply shares already on the edges;
  - the side panel shows the loss, the exit capacity, p* and the paths that carry the loss.
- It reuses 4.3's computation in JavaScript, and becomes more accurate once 4.4's engine and 4.5's slippage curve exist.

4.8 What to implement first

Item	Section	Size	Depends on
Scenario table and p* in `analyze`	4.2	S	—
Shared-node exposure from `graph.json`	4.4	S	—
Hybrid WebSocket trigger	4.6	S	—
`analyze --vault`: depositor loss under a scenario	4.3	M	—
What-if page (sliders, click a node)	4.7	M	4.3
Generic quote source and slippage curve	4.5	M	a keyless aggregator, or an API key
SDK parity check	4.1	S	Node and Morpho's SDK

Item	Section	Size	Depends on
Propagation engine over the graph (DebtRank)	4.4	L	4.5 for realistic liquidations

For the track's scoring grid, the order that moves the most points is 4.3 (impact for the actual use case), then 4.2 (a decision that does not rest on base rates alone), then 4.7 (the demo).

5. Teammate ideas not implemented yet

Ranked by value for the track and by effort, given the existing code.

#	Idea	From	Value	Effort	Fits on top of
1	**Run / queue model for vault exit**: loss as a function of queue position, depositor concentration, φ	Team 2	High: it is the decision for a vault depositor	M	`vault.withdrawable()`, `force_withdrawable()`, unrealised bad debt in `market_risk.py`
2	**Fixed-oracle drain threshold d*** per market, plus the drainable amount (idle + Public Allocator `maxIn` + cap headroom)	Team 2	High: an early, computable exit trigger	S–M	Oracle mode inference, caps already read in `vault.py`; Public Allocator not read yet
3	**Shared-node exposure and alert** (accumulation risk / blast radius)	Team 1, Team 2	High for discovery; directly visible in the graph example	S once there is a shared graph (R3 after R1); a first version can be computed from `graph.json`	`graph.py` output
4	**Scenario library** (named shocks) with the loss per vault depositor	Team 1 / Claude, Team 2 (failure modes)	High: answers "how much do we lose if..."	M	`propagate.loss()` + `market_risk` curves
5	**Push alerts and deduplication**	Team 1 / ChatGPT	High in production	S	Alert objects already complete (next step 1)
6	**Incident replay** at a past block (Stream xUSD / Elixir deUSD, Resolv)	Team 2, Team 1 / Claude	High for credibility	M, needs an archive RPC	Recorded replay infrastructure (next step 5)
7	**Generic aggregator slippage curve** (√ impact) for liquidators and for one's own exit	Team 2	Medium	M	`decide.exit_cost_usd`, next step 2

#	Idea	From	Value	Effort	Fits on top of
8	**Liquidation cascade loop** (selling moves the price and triggers more liquidations)	Team 2, Team 1 / ChatGPT (physics analogy)	Medium	M	`market_risk` curve + item 7
9	**Stuck-time estimate** from the AdaptiveCurveIRM path at 100 % utilisation	Team 2	Medium	S	`borrowRateView` already read
10	**Generic probes**: recursive ERC-4626 unwrapping, Safe threshold, ABI-driven getter crawl	Team 2	Medium: supports the "generalisable" pitch	M (needs an Etherscan key for ABIs)	`admin.py` probes
11	**Portfolio view**: NAV, look-through exposure per root asset	Team 1	Medium-high for Belem	M	Gap M6
12	**Interactive UI**: shock sliders, recolouring the graph	Team 1 / Claude	Medium (demo)	M	`graph.json` as the contract
13	**Partial exit at the marginal optimum**	Team 2	Low-medium	S	`decide.sale_to_reach`
14	Behavioural dependencies (common holders across protocols)	Team 1 / ChatGPT	Low for a POC	L	—
15	LLM fallback for unknown contracts	Team 2 (stretch)	Low	M	—

6. What the implementation has that neither note covers

- **Unrealised bad debt**: underwater borrowers not yet liquidated, with the vault's and the depositor's share. This is the window in which early exits shift losses onto others, the core of Team 2's queue argument, already measured.
- **`forceDeallocate` reach and penalty**, separate from a plain withdrawal: a V2-specific exit path neither note mentions.
- **Vault V2 timelock specifics**: decoded `Submit` with its execution date, abdicated functions, and zero-timelock risk-increasing functions.
- **Admin surface beyond the vault**: oracle and feed upgradeability, collateral pauses, mints of ≥ 5 % of supply (the USR-type incident), and Pendle SY upgrades.
- **Lender concentration** under a borrow: one vault funding ≥ 50 % of a market, and a main lender withdrawing.
- **Data integrity**: block-hash cross-check against an independent RPC, API-vs-chain cross-check, reproducible offline runs with one clock and one cache, and recorded real ranges as

tests.
- **Design traceability**: numbered decisions (D1–D60) in `plan/`, cited in the code; every threshold has a basis in `registry.json`.

7. Corrections to the teammate notes

These points in the notes conflict with Vault V2 or with what was found while building the tool:

- **Team 2 uses MetaMorpho v1 concepts** (`withdrawQueue`, W_0 over the withdraw queue, `lostAssets`). Vault V2 has adapters and a liquidity adapter, not a withdraw queue. Withdrawable = idle + the liquidity adapter's market, and the rest only through `forceDeallocate`. Team 1 / Claude made the same correction.
- **Team 1 / ChatGPT's `maxWithdraw`** returns 0 on Vault V2 (Team 1 / Claude corrected this). The implementation computes exit capacity from the vault's state.
- **Team 2's "Morpho GraphQL for speed, RPC to verify"** matches the implementation. Team 1 / Claude's "chain only, GraphQL only as a dev shortcut" is stricter. The implementation could not list a market's borrowers or a user's positions cheaply on-chain with the free RPC, which refuses old log ranges. That is why it uses the API for discovery.
- **Team 1 / ChatGPT's self-hosted node.** The free RPC's limits (state about 90–126 blocks back, no old logs, 9 addresses per filter) are real. A paid archive RPC is the cheaper next step before a node (next step 6).
- **Team 2's "suppliers aren't liquidated"** is correct and consistent with the model: lenders lose through bad debt and illiquidity, borrowers through liquidation.

8. Questions to settle as a team

1. **Who is the root user for the demo?** A fund depositing in vaults (Team 1, Belem) or a leveraged PT borrower (current `analyze`)? If it is the depositor, items 1, 2 and 4 of section 5 come first.
2. **Probabilities or breakeven?** Keep base-rate Monte Carlo as the headline, or lead with scenarios and p* (Team 2) and show the Monte Carlo as a secondary view?
3. **Shared graph (R1) now or later?** It unlocks shared-node alerts, propagation limits and the "one adapter per protocol" claim, but it is the largest refactor (L).
4. **UI for the demo:** static HTML, Streamlit (stays in Python), or React (Team 1 / Claude)?
5. **Alert channel** (Telegram, ntfy, webhook), so push alerts can ship (S).
6. **Archive RPC key**: needed for incident replay and fork tests, which both notes rate highly for credibility.